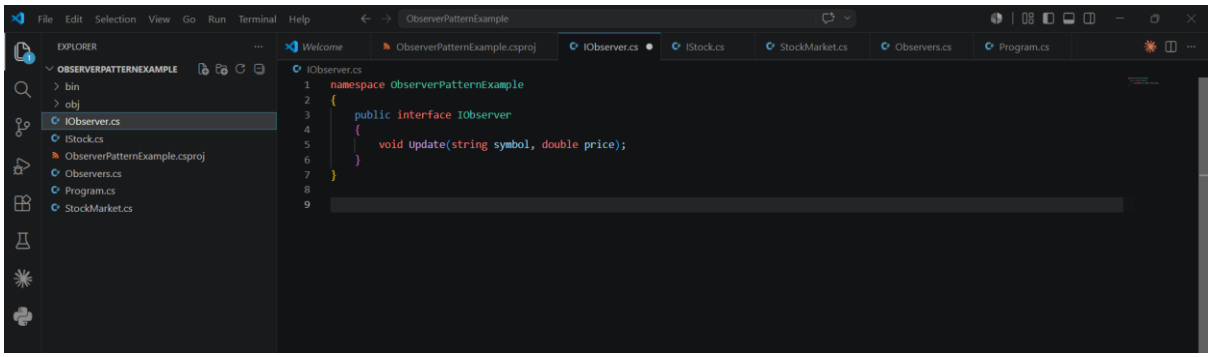
Hands-On Report

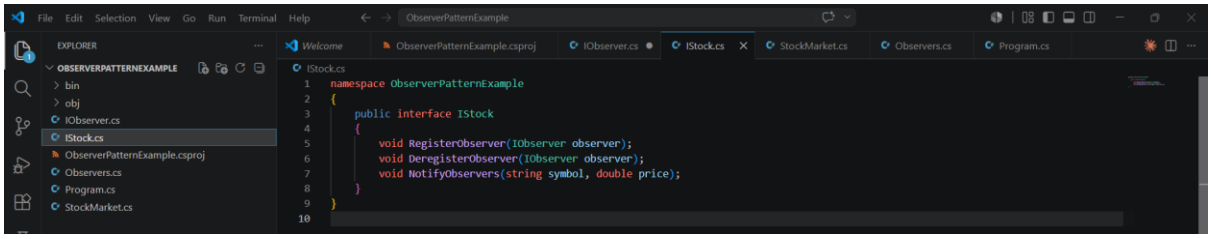
Exercise 7: Implementing the Observer Pattern

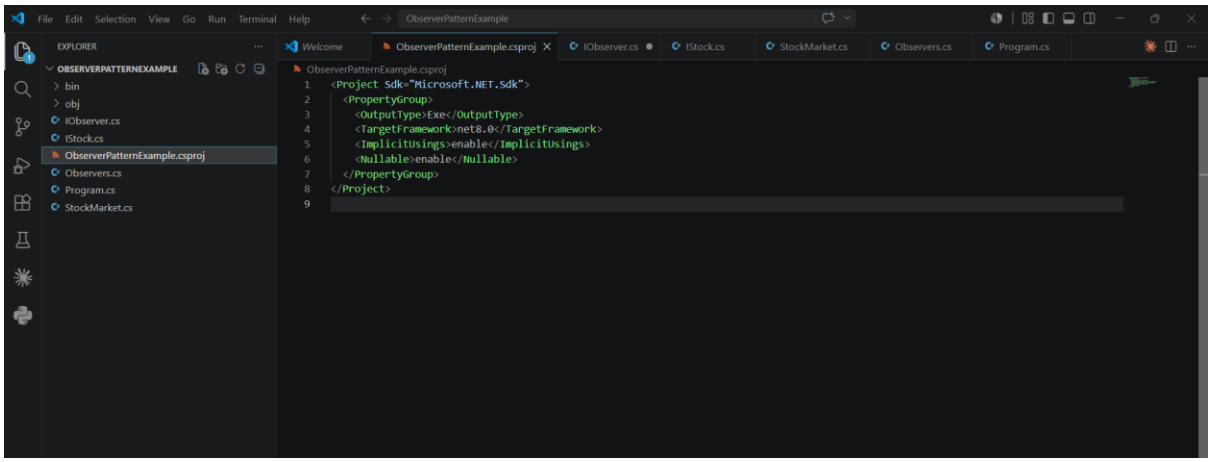
In this exercise, I implemented the Observer Pattern in a .NET application by creating a project named **ObserverPatternExample**. I developed a **Stock** interface to manage observer registration, removal, and notifications, along with a **StockMarket** class that tracked stock price updates. I also created an **Observer** interface and implemented classes such as **MobileApp** and **WebApp** to receive automatic notifications whenever stock information changed.

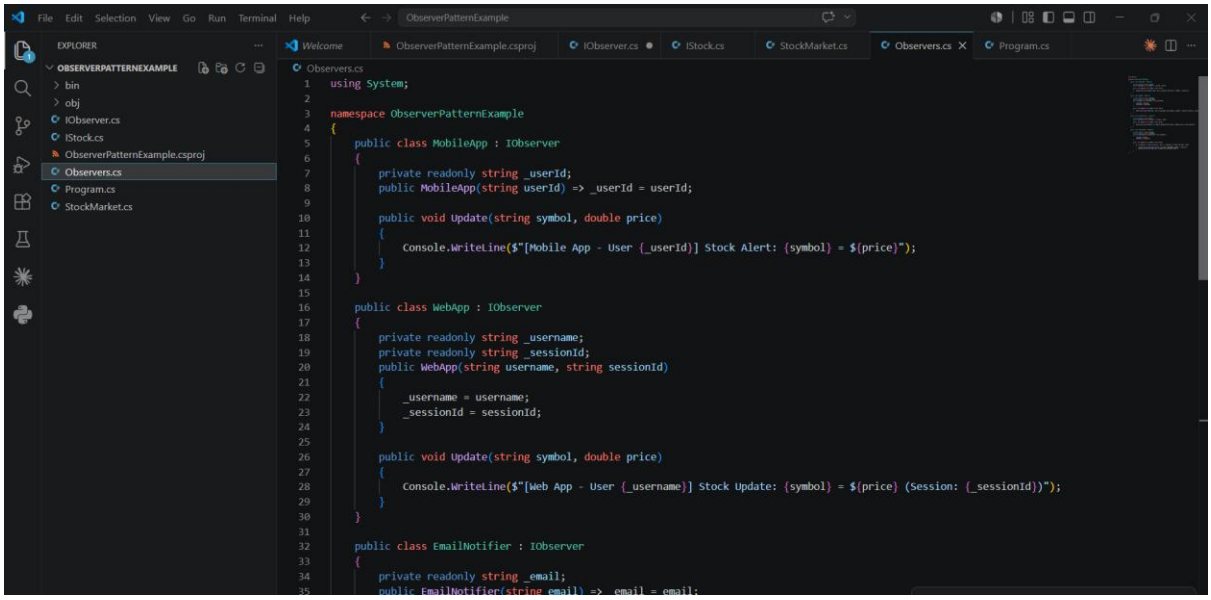
After executing the application, I tested the observer registration, removal, and update process to ensure that all subscribed observers received notifications correctly. This practical exercise helped me understand how the Observer Pattern supports efficient one-to-many communication, reduces dependencies between objects, and keeps related components synchronized in .NET applications.

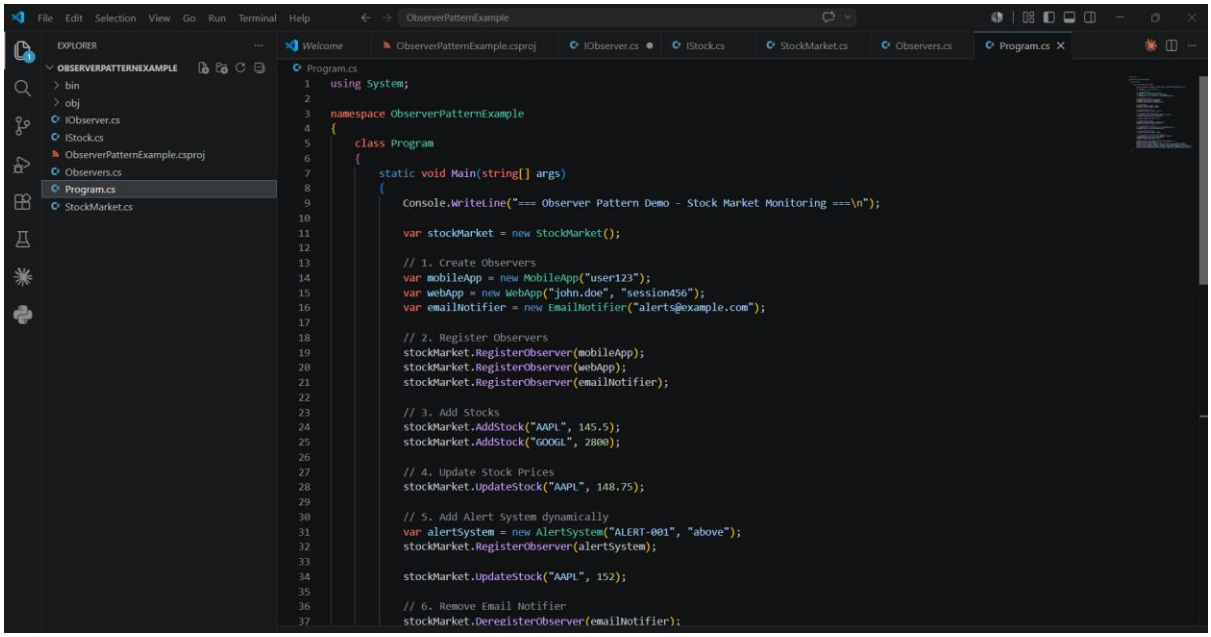
Coding:

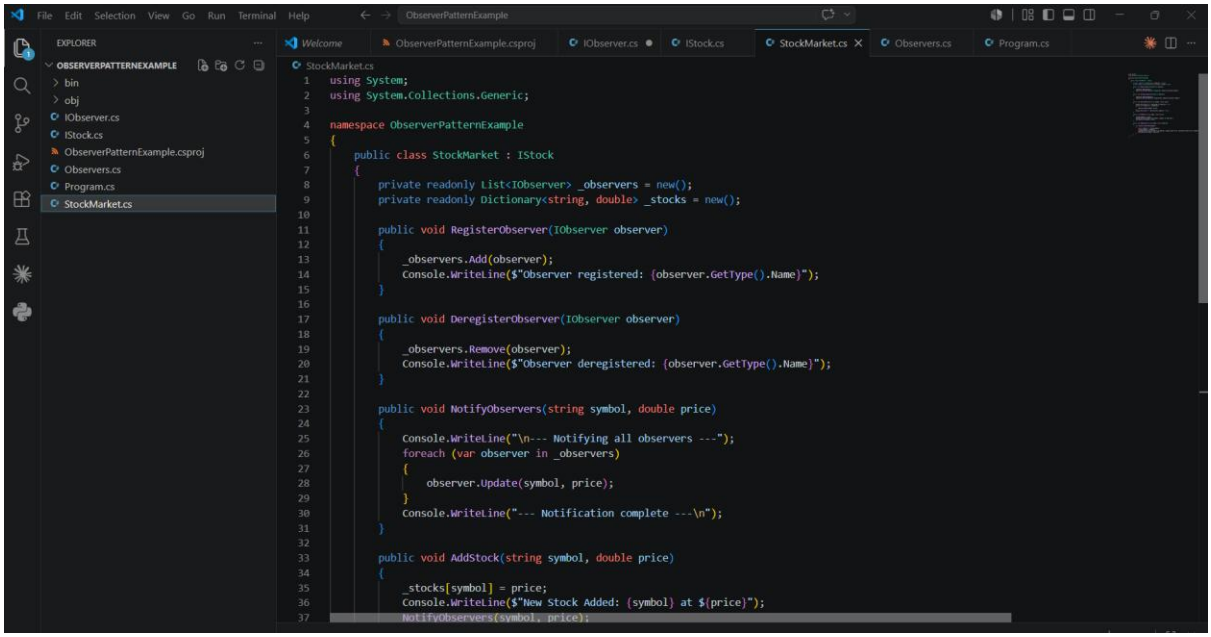


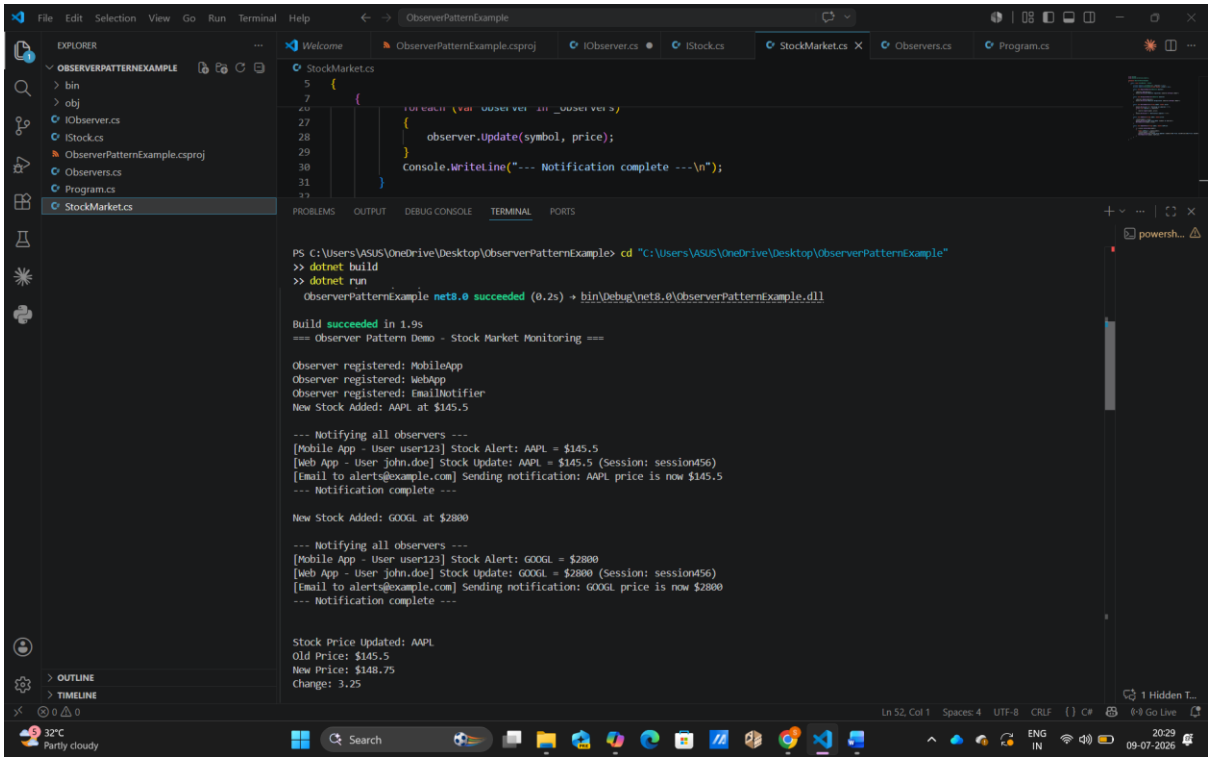

File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXAMPLE
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
Welcome
ObserverPatternExample.csproj
IObserver.cs
IStock.cs
StockMarket.cs
Observers.cs
Program.cs
IStock.cs
namespace ObserverPatternExample
{
public interface IStock
{
void RegisterObserver(IObserver observer);
void DeregisterObserver(IObserver observer);
void NotifyObservers(string symbol, double price);
}
}


File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXAMPLE
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
Welcome
ObserverPatternExample.csproj
IObserver.cs
IStock.cs
StockMarket.cs
Observers.cs
Program.cs
ObserverPatternExample.csproj
<Project Sdk="Microsoft.NET.Sdk">
<PropertyGroup>
<OutputType>Exe</OutputType>
<TargetFramework>net8.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
</PropertyGroup>
</Project>


File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXAMPLE
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
Welcome
ObserverPatternExample.csproj
IObserver.cs
IStock.cs
StockMarket.cs
Observers.cs
Program.cs
Observers.cs
using System;
namespace ObserverPatternExample
{
public class MobileApp : IObserver
{
private readonly string _userId;
public MobileApp(string userId) => _userId = userId;
public void Update(string symbol, double price)
{
Console.WriteLine($"[Mobile App - User {_userId}] Stock Alert: {symbol} = ${price}");
}
}
public class WebApp : IObserver
{
private readonly string _username;
private readonly string _sessionId;
public WebApp(string username, string sessionId)
{
_username = username;
_sessionId = sessionId;
}
public void Update(string symbol, double price)
{
Console.WriteLine($"[Web App - User {_username}] Stock Update: {symbol} = ${price} (Session: {_sessionId})");
}
}
public class EmailNotifier : IObserver
{
private readonly string _email;


File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXAMPLE
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
Welcome
ObserverPatternExample.csproj
IObserver.cs
IStock.cs
StockMarket.cs
Observers.cs
Program.cs
Program.cs
using System;
namespace ObserverPatternExample
{
class Program
{
static void Main(string[] args)
{
Console.WriteLine("=== Observer Pattern Demo - Stock Market Monitoring ===\n");
var stockMarket = new StockMarket();
// 1. Create Observers
var mobileApp = new MobileApp("user123");
var webApp = new WebApp("john.doe", "session456");
var emailNotifier = new EmailNotifier("alerts@example.com");
// 2. Register Observers
stockMarket.RegisterObserver(mobileApp);
stockMarket.RegisterObserver(webApp);
stockMarket.RegisterObserver(emailNotifier);
// 3. Add Stocks
stockMarket.AddStock("AAPL", 145.5);
stockMarket.AddStock("GOOGL", 2800);
// 4. Update Stock Prices
stockMarket.UpdateStock("AAPL", 148.75);
// 5. Add Alert System dynamically
var alertSystem = new AlertSystem("ALERT-001", "above");
stockMarket.RegisterObserver(alertSystem);
stockMarket.UpdateStock("AAPL", 152);
// 6. Remove Email Notifier
stockMarket.DeregisterObserver(emailNotifier);


File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXAMPLE
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
Welcome
ObserverPatternExample.csproj
IObserver.cs
IStock.cs
StockMarket.cs
Observers.cs
Program.cs
StockMarket.cs
using System;
using System.Collections.Generic;
namespace ObserverPatternExample
{
public class StockMarket : IStock
{
private readonly List<IObserver> _observers = new();
private readonly Dictionary<string, double> _stocks = new();
public void RegisterObserver(IObserver observer)
{
_observers.Add(observer);
Console.WriteLine($"Observer registered: {observer.GetType().Name}");
}
public void DeregisterObserver(IObserver observer)
{
_observers.Remove(observer);
Console.WriteLine($"Observer deregistered: {observer.GetType().Name}");
}
public void NotifyObservers(string symbol, double price)
{
Console.WriteLine("\n--- Notifying all observers ---");
foreach (var observer in _observers)
{
observer.Update(symbol, price);
}
Console.WriteLine("--- Notification complete ---\n");
}
public void AddStock(string symbol, double price)
{
_stocks[symbol] = price;
Console.WriteLine($"New Stock Added: {symbol} at ${price}");
NotifyObservers(symbol, price);


File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXAMPLE
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
Welcome
ObserverPatternExample.csproj
IObserver.cs
IStock.cs
StockMarket.cs
Observers.cs
Program.cs
StockMarket.cs
{
{
observer.Update(symbol, price);
}
Console.WriteLine("--- Notification complete ---\n");
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powersh...
PS C:\Users\ASUS\OneDrive\Desktop\ObserverPatternExample> cd "C:\Users\ASUS\OneDrive\Desktop\ObserverPatternExample"
>> dotnet build
>> dotnet run
ObserverPatternExample net8.0 succeeded (0.2s) → bin\Debug\net8.0\ObserverPatternExample.dll
Build succeeded in 1.9s
=== Observer Pattern Demo - Stock Market Monitoring ===
Observer registered: MobileApp
Observer registered: WebApp
Observer registered: EmailNotifier
New Stock Added: AAPL at $145.5
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $145.5
[Web App - User john.doe] Stock Update: AAPL = $145.5 (Session: session456)
[Email to alerts@example.com] Sending notification: AAPL price is now $145.5
--- Notification complete ---
New Stock Added: GOOGL at $2800
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: GOOGL = $2800
[Web App - User john.doe] Stock Update: GOOGL = $2800 (Session: session456)
[Email to alerts@example.com] Sending notification: GOOGL price is now $2800
--- Notification complete ---
Stock Price Updated: AAPL
Old Price: $145.5
New Price: $148.75
Change: 3.25
OUTLINE
TIMELINE
Ln 52, Col 1
Spaces: 4
UTF-8
CRLF
C#
Go Live
1 Hidden T...
32°C
Partly cloudy
Search
ENG
IN
20:29
09-07-2026